



Programa Certified AI-LLM Integration Engineer

\Curso de Fundamentos de LLMs y APIs

Aplicación de Sistemas Distribuidos y Contenedores en Infraestructura Escalable para
LLMs

www.bsginstitute.com

Andrés Felipe Rojas Parra

- Ingeniero electrónico con mas de 23 años de experiencia
- PGP Artificial Intelligence & Machine Learning at Texas University – Austin, TX
- Master en Inteligencia Artificial y Big Data – IEBSchool
- Estudiante de PGP en Cloud Computing at Texas University - Austin, TX
- Texas Executive Education Program – McCombs School of Business – Decision Science and AI Immersion Program
- Especialización en MLOps de la Universidad de Duke



arojaspa@outlook.com



@arojaspa



<https://www.linkedin.com/in/arojaspa/>



+573005906373



Fundamentos de Arquitectura LLM

Capítulo 1 — Sesión 1: Conceptos Fundamentales

Tema 1: Introducción a Arquitecturas LLM

Tema 2: Tipos de Modelos y Servicios

Tema 3: Consideraciones de Costos

Agenda de la Sesión

1

Introducción a Arquitecturas LLM

~60 min

- Bienvenida e introducción
- Origen y evolución de los LLM
- Relevancia en el ecosistema IA
- Impacto real en la industria

2

Tipos de Modelos y Servicios

~60 min

- Decoder, Encoder, Encoder-Decoder
- Cloud vs On-premise
- Casos de uso LATAM
- Comparativa de proveedores

3

Consideraciones de Costos

~60 min

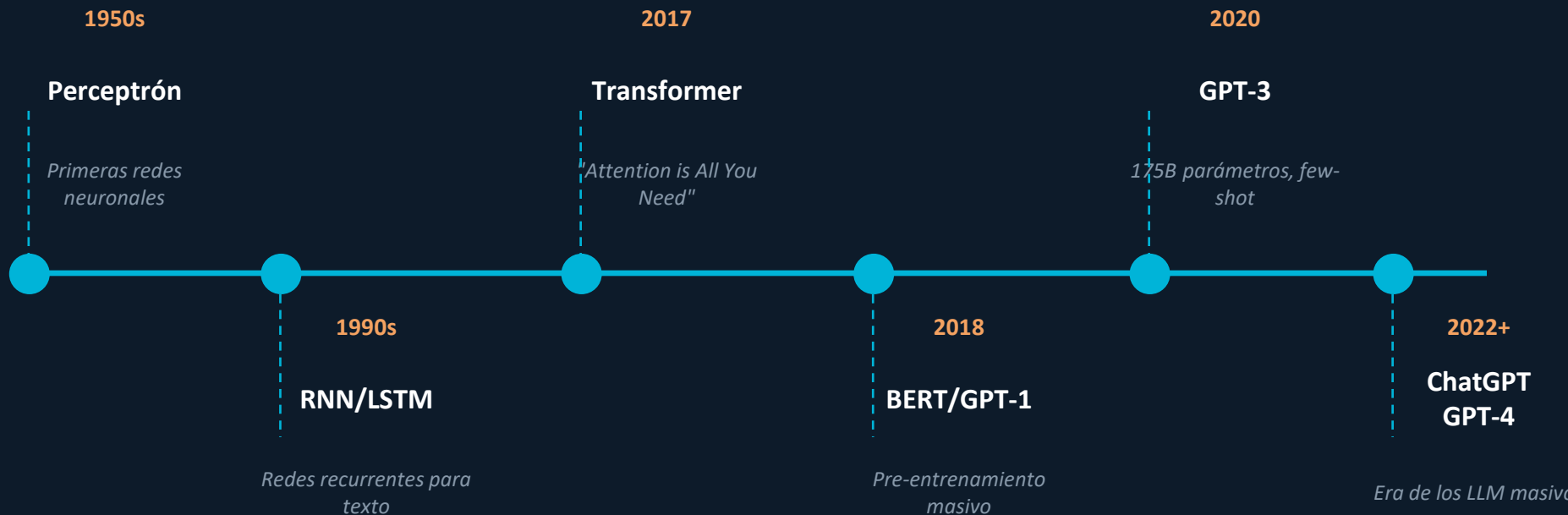
- Evaluación de proveedores
- Costos a corto y largo plazo
- ROI y análisis de riesgo
- Demo: calculadora de costos

TEMA 1

Introducción a Arquitecturas LLM

Del perceptrón al GPT-4 en 7 décadas de historia

Evolución de los LLM



🔑 El paper "Attention is All You Need" (Google, 2017) es el punto de inflexión: introduce la arquitectura Transformer que sustenta todos los LLM modernos.

El Perceptrón — Frank Rosenblatt, 1958

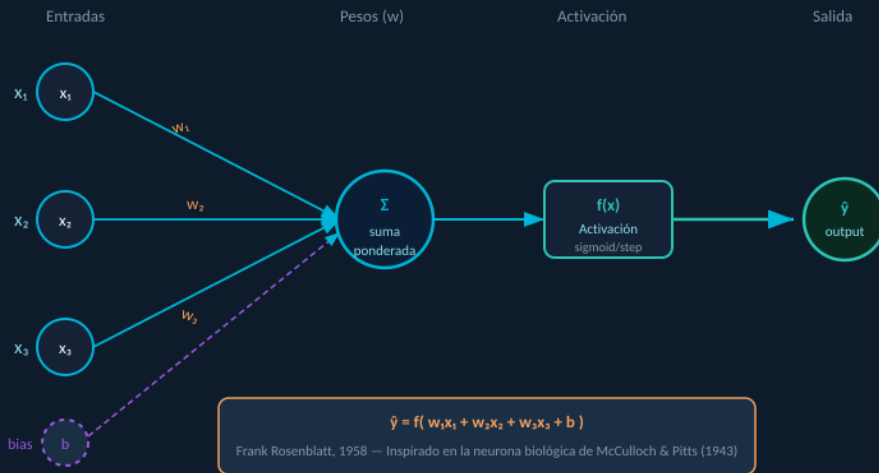
¿Qué es el Perceptrón?

La unidad mínima de una red neuronal. Inspirado en la neurona biológica, recibe múltiples entradas, las pondera con pesos y aplica una función de activación para producir una salida.

Rosenblatt construyó el Mark I Perceptron en hardware en 1958 para reconocer imágenes. Era capaz de distinguir formas simples con una precisión sorprendente para la época.

Conceptos clave:

- **Pesos (w):** importancia de cada entrada
- **Bias (b):** umbral de activación
- **Activación:** decide si la neurona 'dispara'
- **Backprop:** ajusta pesos por error (1986)



→ Múltiples capas = Red Neuronal (MLP)



RNN y LSTM — Memoria en Secuencias (1990s)

¿Por qué RNN?

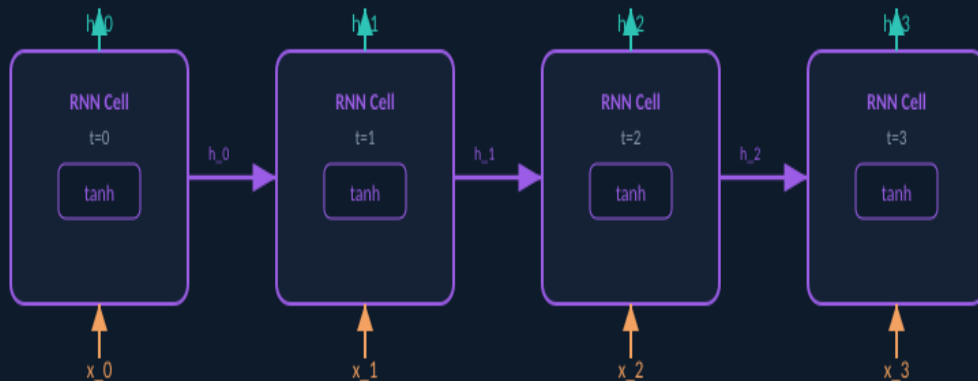
Los MLP procesan entrada de tamaño fijo. Pero el lenguaje es secuencial: el significado de una palabra depende de las anteriores. Las RNN introducen memoria al pasar un estado oculto (h_t) de un paso al siguiente.

Problema: las RNN clásicas 'olvidan' información lejana (vanishing gradient).

LSTM — Solución (1997):

- **Forget Gate:** ¿qué olvidar?
- **Input Gate:** ¿qué guardar?
- **Output Gate:** ¿qué comunicar?
- **Limitación:** secuencial → lento a escala

RNN — Red Neuronal Recurrente (unrolled / desplegada en el tiempo)



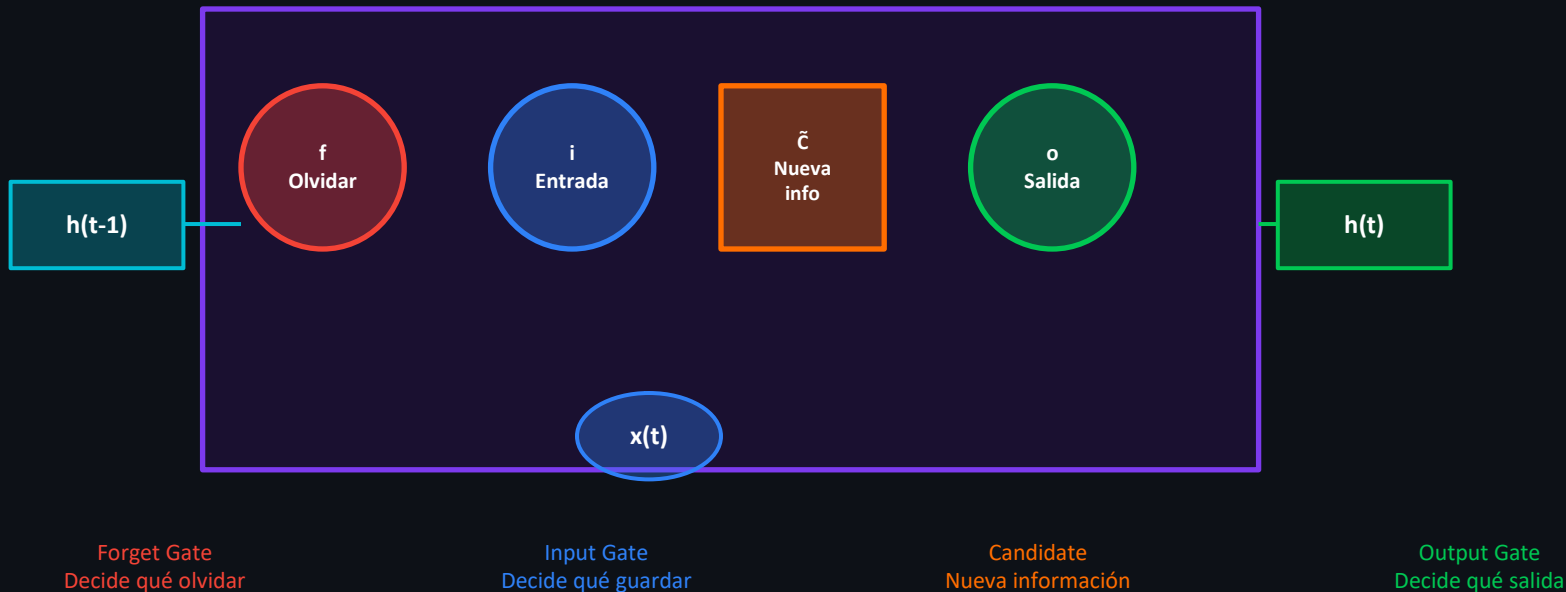
LSTM — Long Short-Term Memory: Puertas que controlan la memoria



Long Short-Term Memory (LSTM)

Inventado por Hochreiter & Schmidhuber (1997) — Soluciona el vanishing gradient con celdas de memoria

$C(t)$ Estado de Celda (memoria de largo plazo)



Por qué
LSTM
funciona:

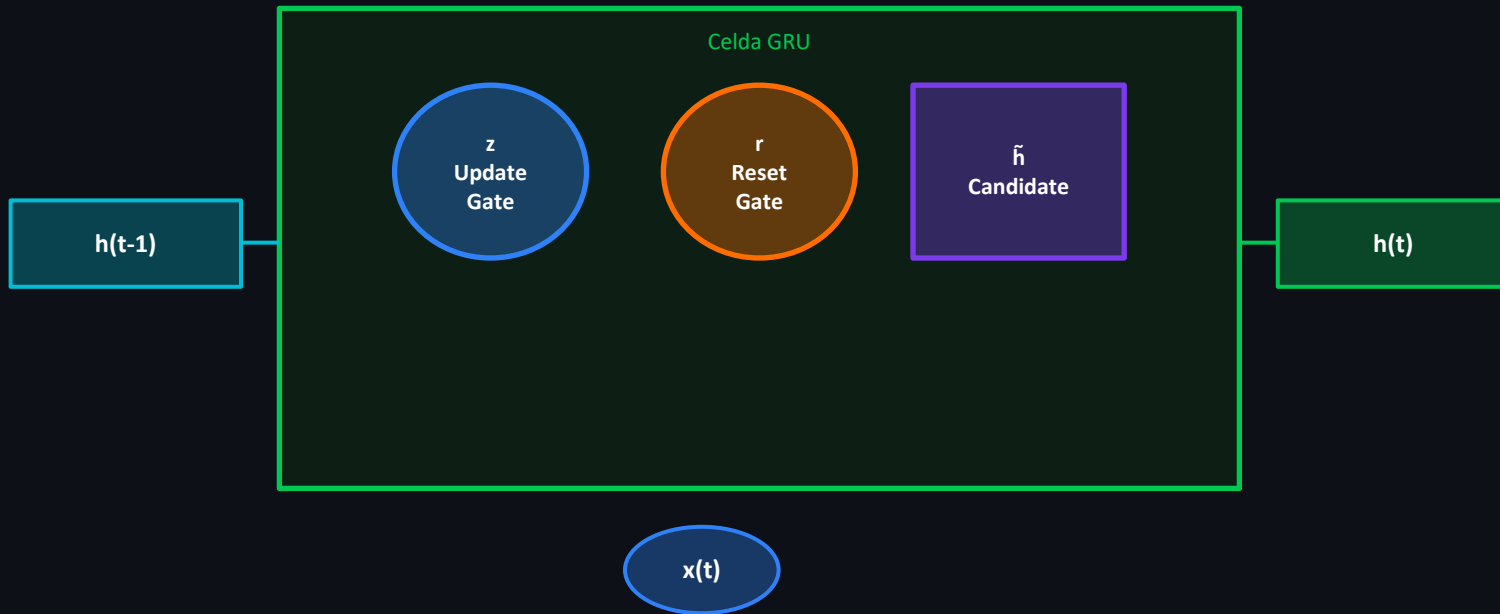
✓
Memoria
long-term

✓ Control
fino de
q olvidar

✓
Gradiente
fluye sin
desaparecer

Gated Recurrent Unit (GRU)

Propuesto por Cho et al. (2014) — Versión simplificada y más eficiente del LSTM



GRU vs LSTM:

GRU: 2 puertas

LSTM: 3 puertas

GRU: más rápido

LSTM: más preciso

GRU: menos params

Similar desempeño

El Transformer — Attention Is All You Need (Google Brain, 2017)

El paper que lo cambió todo

Vaswani et al. propusieron eliminar la recurrencia completamente. La idea central:

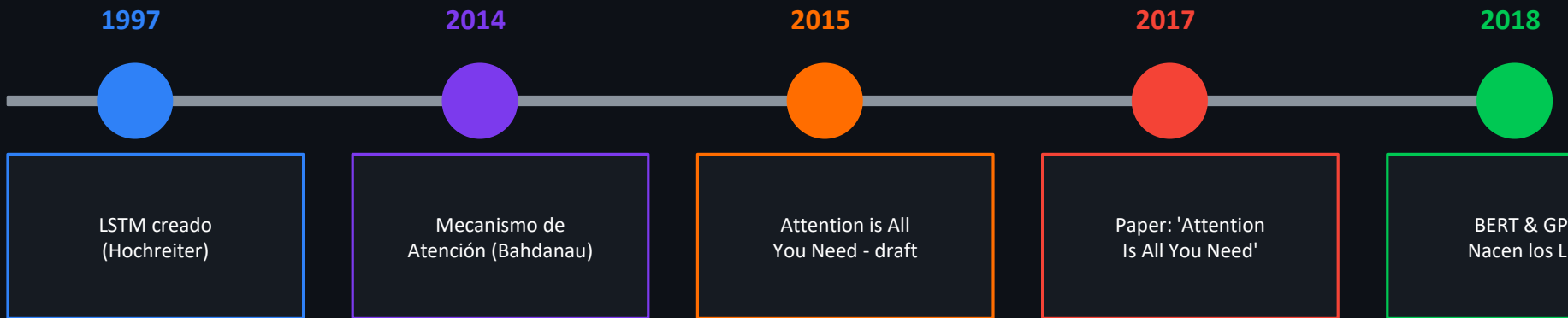
Self-Attention: cada token puede atender a todos los demás simultáneamente. Sin memoria secuencial → paralelizable al 100%.

Por qué es revolucionario:

- **GPU paralela:** entrena 10-100x más rápido
- **Contexto global:** atiende toda la oración
- **Escalable:** más datos → mejor modelo
- **Transfer learning:** pre-entrenar + fine-tune



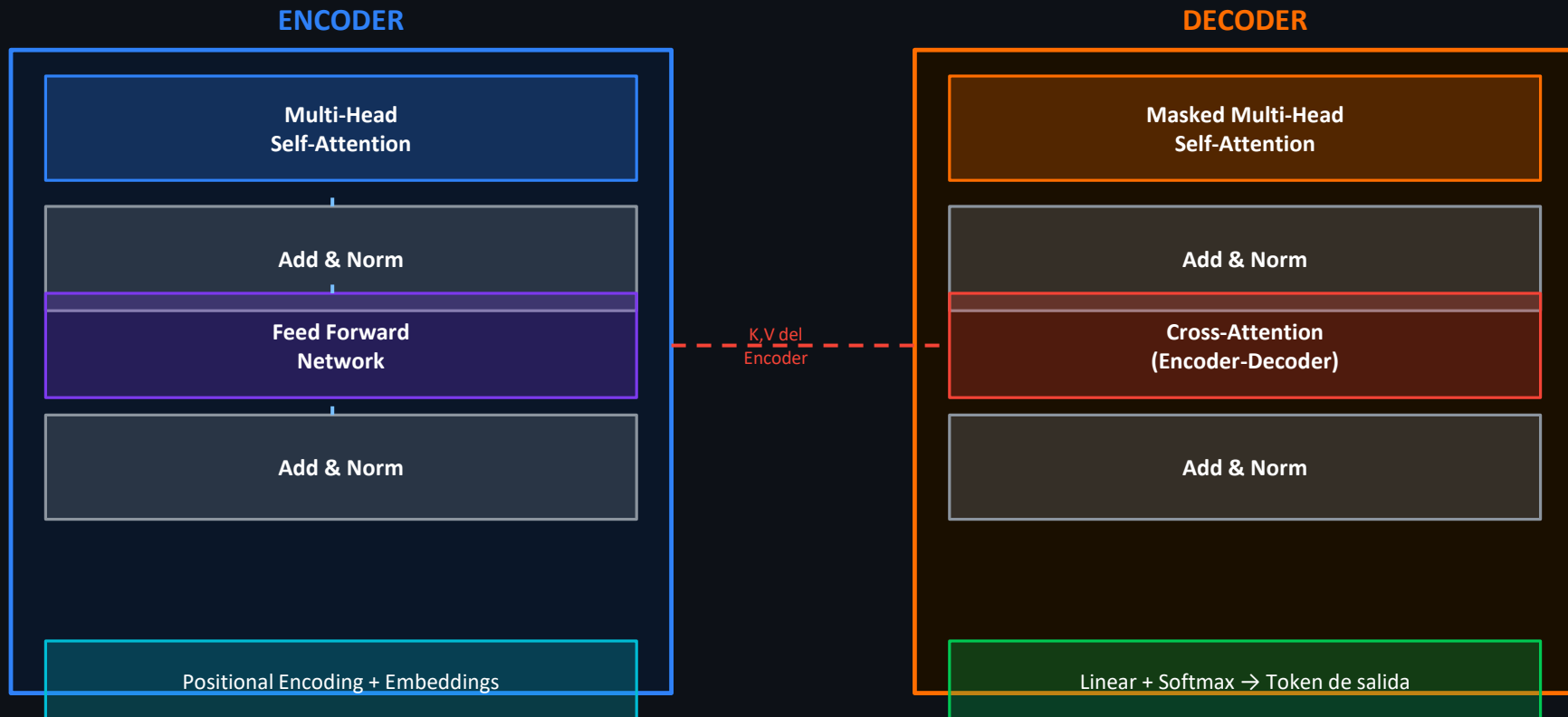
El Nacimiento de los Transformers



La Idea Central: Mecanismo de Atención (Self-Attention)

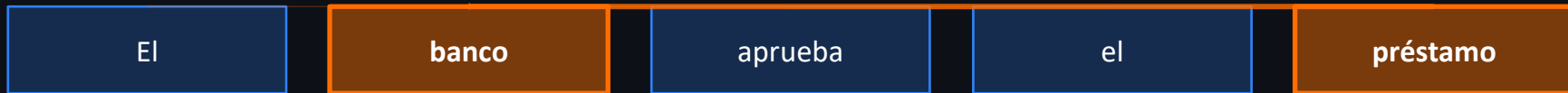
En lugar de procesar palabras secuencialmente (como RNN), los Transformers miran TODAS las palabras al mismo tiempo y calculan qué tan relevante es cada palabra para entender cada otra palabra — como un 'spotlight' que ilumina las conexiones importantes.

Arquitectura del Transformer



Mecanismo de Self-Attention (Atención)

Ejemplo: "El banco aprueba el préstamo"



¿Cómo funciona internamente?



Query (Consulta)

¿Qué estoy buscando?
"banco" pregunta por
su contexto



Key (Clave)

¿Qué tengo para ofrecer?
Cada palabra anuncia
su contenido



Value (Valor)

La información real
que se transmite si
hay match

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

BERT (Google) y GPT-1 (OpenAI) — La carrera comienza, 2018

Ambos modelos nacen en 2018 usando el mismo Transformer, pero con objetivos opuestos. Google quería comprender texto → BERT. OpenAI quería generar texto → GPT-1. Esta divergencia define dos linajes que coexisten hasta hoy.

BERT (Google, Oct 2018)

Bidirectional Encoder Representations from Transformers

Pre-entrenamiento BIDIRECCIONAL:



↔ Ve contexto completo en ambas direcciones

Pre-entrenamiento: MLM + NSP

Parámetros:

- BERT-base: 110M params, 12 capas
- BERT-large: 340M params, 24 capas

Ideal para:

- Clasificación, NER, embeddings
- Búsqueda semántica (RAG)
- Sentiment analysis

Devlin, Chang, Lee & Toutanova — Google AI Language

GPT-1 (OpenAI, Jun 2018)

Generative Pre-trained Transformer

Pre-entrenamiento UNIDIRECCIONAL (causal):



→ Solo ve tokens anteriores (masked)

Pre-entrenamiento: Next Token Prediction

Parámetros:

- 117M parámetros, 12 capas
- BooksCorpus: 7,000 libros no publicados

Ideal para:

- Generación de texto
- Completar frases (autoregresivo)
- Base de todos los GPT posteriores

Radford, Narasimhan, Salimans, Sutskever — OpenAI

GPT-3 — Ley de Escala: 'Más datos + más params = capacidades emergentes' (OpenAI, 2020)

"Language Models are Few-Shot Learners" — Brown et al. La hipótesis: si escalamos el modelo lo suficiente, emergen capacidades nuevas sin necesidad de fine-tuning. GPT-3 lo demostró con 175B parámetros.

Escala sin precedentes: La ley de escalamiento cambia todo



Brown et al. (2020) — "Language Models are Few-Shot Learners" — OpenAI. Paper que demostró que escala = capacidad emergente.

ChatGPT y GPT-4 — El LLM llega al mundo (OpenAI, 2022–2023)

ChatGPT no fue un modelo nuevo: fue un cambio de paradigma en cómo se interactúa con un LLM. GPT-3.5 + RLHF + interfaz conversacional = 100 millones de usuarios en 60 días, el producto de mayor crecimiento en la historia.

Del modelo al producto: cómo nació ChatGPT y GPT-4



¿Qué es RLHF? (Clave de ChatGPT)

- 1 Supervised Fine-Tuning**
Humanos escriben respuestas ideales → modelo aprende
- 2 Reward Model**
Humanos rankan respuestas → modelo aprende qué es 'bueno'
- 3 PPO (Reinforcement)**
Modelo se refuerza maximizando el reward score

GPT-4 — Capacidades sin precedente

- Multimodal:** Procesa imágenes y texto simultáneamente
- Razonamiento:** Bar exam top 10%, SAT 1300+, coding avanzado
- Contexto 128K:** ~100,000 palabras en un solo prompt
- Function Calling:** Llama APIs externas desde el chat
- Safety:** RLHF + Constitutional AI approach

Inteligencia Artificial Generativa

IA que no solo analiza datos, sino que CREA contenido nuevo: texto, imágenes, audio, código, video...



GANs

Generative Adversarial
Networks (2014)

Generador vs Discriminador
en competencia

► **Imágenes realistas · Deepfakes**



VAEs

Variational
AutoEncoders (2013)

Comprimen y reconstruyen
datos en espacio latente

► **Síntesis de datos · Anomaly detection**



Diffusion

Modelos de Difusión
(2020-2022)

Aprenden a revertir
el proceso de ruido

► **DALL-E, Stable Diffusion · Midjourney**



LLMs

Large Language
Models (2018-hoy)

Transformers entrenados
en texto masivo

► **ChatGPT, Claude, · Gemini, Llama**

Large Language Models (LLMs)

Un LLM es un modelo de lenguaje con miles de millones de parámetros, entrenado sobre enormes corpus de texto usando la arquitectura Transformer, capaz de generar, resumir, traducir y razonar sobre texto.

Etapas de entrenamiento de un LLM:



Modelos destacados:

GPT-4

Claude 3

Gemini

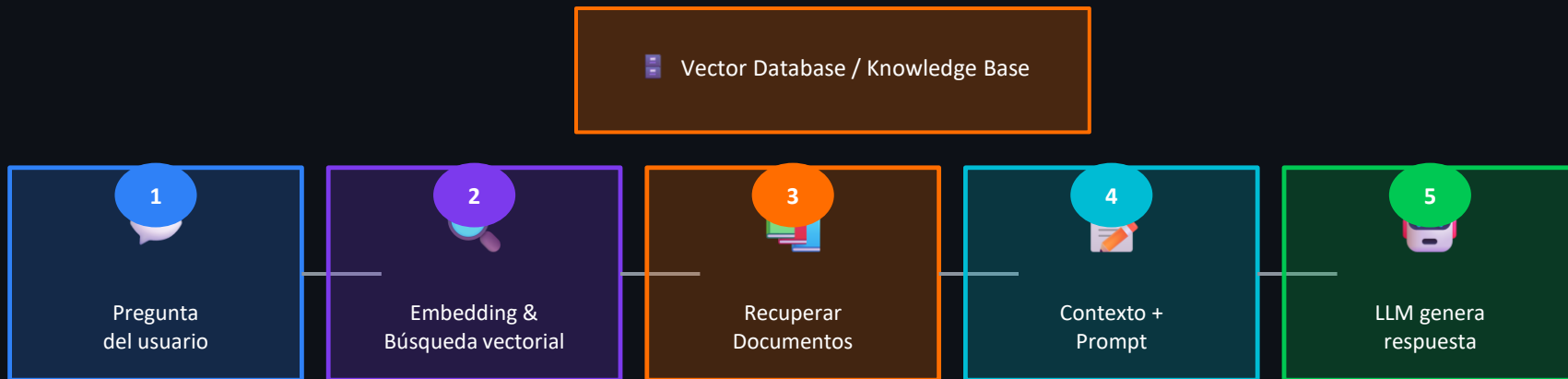
Llama 3

Mistral

Command R

Retrieval-Augmented Generation (RAG)

Técnica que combina un LLM con una base de conocimiento externa para reducir alucinaciones y usar información actualizada



✗ Sin RAG (LLM solo)

- Alucinaciones frecuentes
- Información desactualizada
- Sin acceso a datos privados
- Costoso re-entrenar

✓ Con RAG

- Respuestas verificables con fuentes
- Información siempre actualizada
- Acceso a documentos privados
- Flexible y eficiente

La Nueva Era de los LLM Masivos — 2024 y más allá

Ya no es una carrera solo de parámetros: es una carrera de razonamiento, multimodalidad, costo y acceso. El open source alcanza a los modelos propietarios. LATAM tiene un rol activo como mercado y como generador de casos de uso.

El Ecosistema LLM 2024-2025: Competencia global y open source

OpenAI

- GPT-4o (multimodal)
- o1/o3 (razonamiento)
- GPT-4.1 nano/mini

Microsoft \$13B invertido

Anthropic

- Claude 3.7 Sonnet
- Claude 3.5 Haiku
- Constitutional AI

Amazon \$4B + Google \$300M

Google DeepMind

- Gemini 2.0 Flash
- Gemini 2.5 Pro
- 1M token context

Integrado en GCP/Workspace

Meta AI (Open)

- Llama 3.1/3.2/3.3
- 405B parámetros
- Ollama / HuggingFace

Open weights — free

Tendencias que definen la Nueva Era:

Razonamiento extendido

o1/o3, DeepSeek-R1: los modelos piensan antes de responder (Chain-of-Thought nativo)

Multimodalidad nativa

Texto + imagen + audio + video en el mismo modelo. GPT-4o, Gemini 2.0.

Contextos ultra-largos

1M+ tokens. Gemini 2.5 Pro: libro completo + código + documentación en un solo prompt.

Agentes autónomos (Agentic AI)

Modelos que planifican, usan herramientas y ejecutan tareas multi-paso sin supervisión.

Open Source vs Propietario

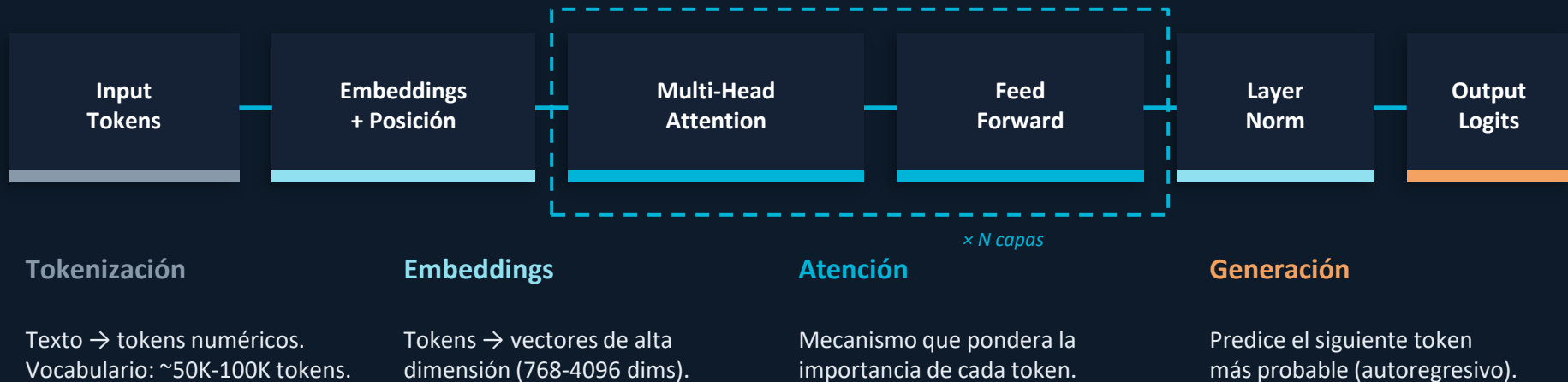
Llama 3, Mistral, Qwen2.5, DeepSeek-R1: calidad que rivaliza con modelos cerrados.

Modelos multilingüe LATAM

Español nativo, Guaraní, Quechua. OpenAI, Google y startups locales invirtiendo en LATAM.

2025: Inversión global en AI supera \$300B anuales. LATAM: \$8B+ en adopción empresarial proyectada para 2027.

¿Qué hay dentro de un LLM?

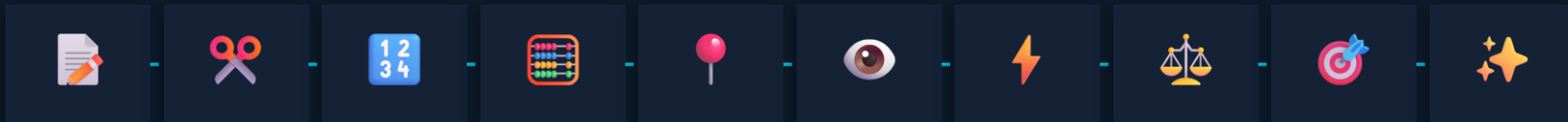


GPT-4 tiene estimados ~1.8 BILLONES de parámetros organizados en cientos de capas como la mostrada arriba. Cada parámetro es un número que el modelo aprendió de trillones de palabras.

D E E P D I V E

El Pipeline del Transformer

Tokenización → Embeddings → Atención → Feed Forward → Generación



6 slides — Cada componente explicado con diagrama original

① Tokenización — El texto se convierte en números

¿Qué es un token?

Un token es la unidad mínima que el modelo procesa. No es una palabra: puede ser parte de una palabra, una palabra entera o un signo de puntuación.

BPE (Byte-Pair Encoding)

Algoritmo que combina pares frecuentes. GPT-4 y Llama lo usan.
Vocabulario: ~100K.

WordPiece

Variante de BPE usada en BERT.
Fragmenta en sub-palabras con prefijo ## para continuaciones.

Costo real

1 token $\approx$ 0.75 palabras (inglés). En español $\approx$ 0.6 palabras. Más tokens = mayor costo en APIs.

① Texto crudo (raw text)

"El gato duerme"

WordPiece / BPE tokenizer

② Tokens (subword / WordPiece)

El

palabra completa

ga

sub-word (##)

##to

sub-word (##)

duer

sub-word (##)

vocab lookup (índice en vocabulario)

③ Token IDs (enteros)

2590

18291

1102

14822

embedding lookup table

④ Vectores de embedding (d=768 dimensiones)

[0.23, -0.87, 0.45,
1.02, -0.31, ...
768 floats

[0.23, -0.87, 0.45,
1.02, -0.31, ...
768 floats

[0.23, -0.87, 0.45,
1.02, -0.31, ...
768 floats

[0.23, -0.87, 0.45,
1.02, -0.31, ...
768 floats

Vocabulario
~30K-100K tokens

GPT-4: ~100K tokens
BERT: 30K tokens
Llama3: 128K tokens
mT5: 250K tokens
Mistral: 32K tokens

BPE, WordPiece,
SentencePiece
(algoritmos)

1 token $\approx$ 0.75 palabras

Los tokens desconocidos se fragmentan en sub-palabras conocidas. Ej: "tokenización" $\rightarrow$ ["token", "##ización"]

② Embeddings + Positional Encoding — De número a vector con posición

Token Embedding

Tabla lookup: cada token ID mapea a un vector denso de d dimensiones. El modelo aprende estos vectores durante el entrenamiento.

¿Qué captura?

Semántica: 'rey' - 'hombre' + 'mujer' $\approx$ 'reina'. Relaciones sintácticas y conceptuales en un espacio vectorial.

Positional Encoding

Los Transformers no tienen noción de orden (no son RNN). El PE agrega una señal de posición única por posición.

PE fijo vs learnable

Vaswani: sin/cos fijo. GPT-2: learnable. RoPE (Llama): rotatorio. ALiBi (MPT): sesgos.

Token Embedding

Semántica de la palabra



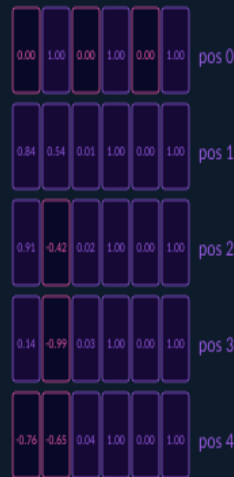
... 768 dimensiones ...



suma
elemento
a elemento

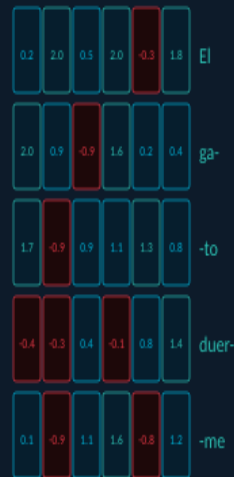
Positional Encoding

Posición en la oración



Input al Transformer

Semántica + Posición



$$PE(pos, 2i) = \sin(pos/10000^{2i/4})$$

$$PE(pos, 2i+1) = \cos(pos/10000^{2i/4})$$

Sin PE: "gato come ratón" = "ratón come gato"

El PE da al modelo sentido del orden
sin parámetros adicionales entrenables

③ Multi-Head Self-Attention — El corazón del Transformer

Self-Attention

Cada token mira a TODOS los demás tokens simultáneamente y decide cuánta atención darles. Sin restricción de distancia.

Q, K, V

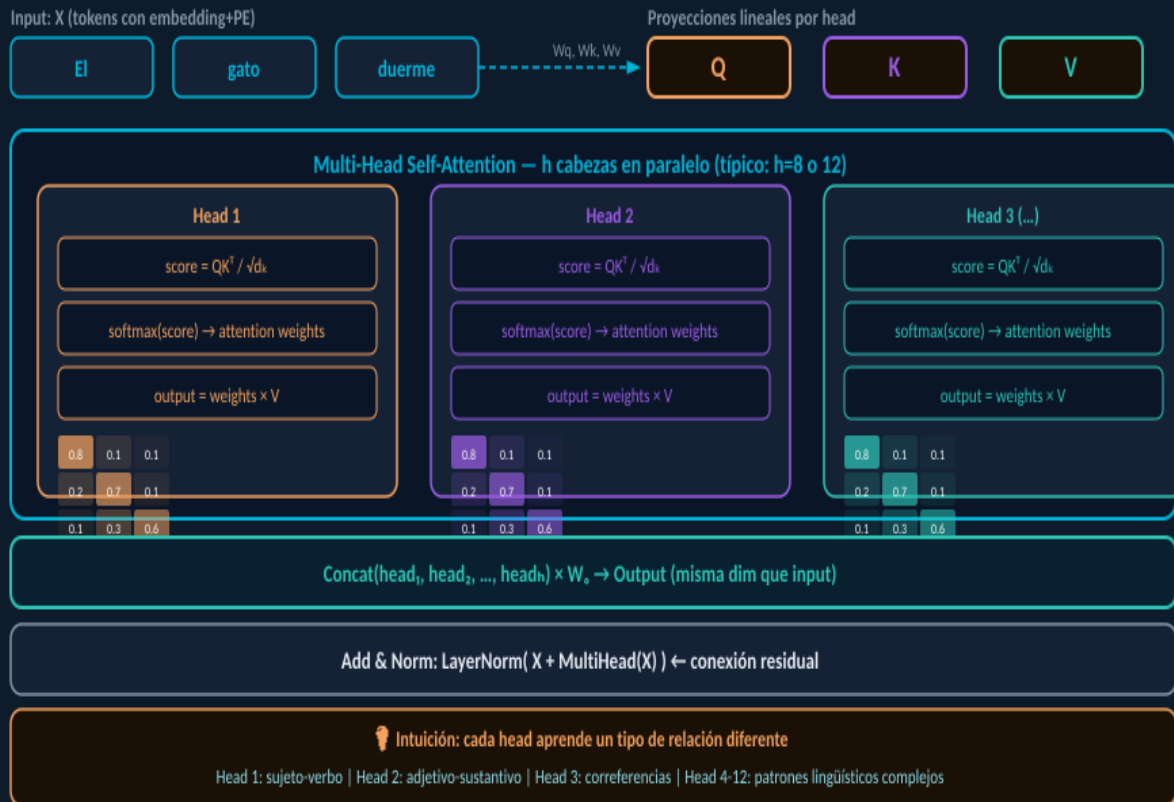
Query: '¿qué busco?' Key: '¿qué tengo?'
Value: 'mi contenido'. Score = $Q \cdot K^T / \sqrt{d_k}$

Softmax → Weights

Los scores se convierten en probabilidades que suman 1. Eso define cuánto de cada V tomar.

Multi-Head

Múltiples copias en paralelo. Cada head aprende un tipo distinto de relación. Sus outputs se concatenan.



④ Feed Forward Network — El 'conocimiento' del modelo

Después del FFN

Después de que la atención mezcla información entre tokens, el FFN procesa cada token independientemente para transformar su representación.

Expand × 4

El FFN expande la dimensión por 4 (768 → 3072 en BERT, 4096 → 16384 en GPT-3). Más espacio = más expresividad.

GELU / ReLU

GELU es la activación estándar en transformers modernos. Suave y diferenciable. ReLU en modelos más antiguos.

¿Dónde está el 'conocimiento'?

Investigaciones muestran que los hechos (París capital de Francia) están almacenados en los pesos del FFN, no en la atención.

Feed Forward Network (FFN) — Razonamiento posición a posición

Para cada token independientemente:



MHA vs FFN — Roles complementarios

Multi-Head Attention

¿Cómo los tokens se relacionan entre sí?

Mira el contexto GLOBAL

Params: $4 \times d^2$ (Q,K,V,O matrices)

~% total: ~33% del modelo

Feed Forward Network

¿Qué hace el modelo con cada token?

Razonamiento LOCAL por posición

Params: $2 \times d \times 4d$ (expand/compress)

~% total: ~67% del modelo

GPT-3: 96 capas × (MHA + FFN) = 192 sub-bloques

El 67% de los parámetros está en las FFN (el "conocimiento" del modelo)

⑤ Layer Norm + Conexión Residual — Estabilidad en 96 capas

Problema sin normalización

En redes profundas (96 capas), las activaciones exploran o se desvanecen. Imposible entrenar sin normalización.

Layer Norm

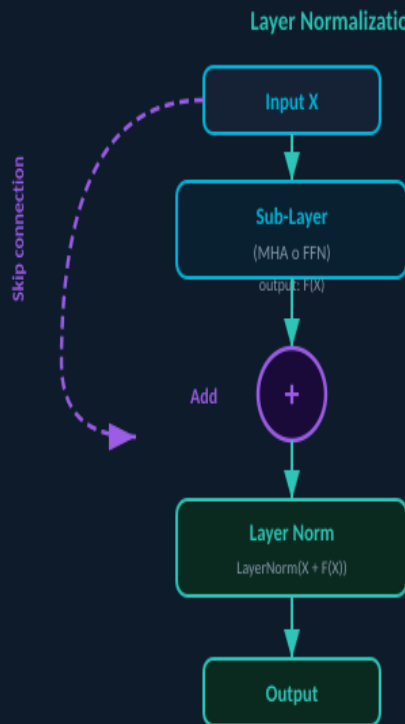
Normaliza cada vector a $\mu=0$, $\sigma=1$ a lo largo de las dimensiones del embedding. Luego escala con γ y β aprendibles.

vs Batch Norm

Batch Norm normaliza por batch (malo para secuencias variables). Layer Norm normaliza por ejemplo → ideal para NLP.

Conexión residual

El input X se suma al output de la sub-capa: $\text{LayerNorm}(X + \text{sublayer}(X))$. Permite que el gradiente fluya directo.



$$\text{output} = \text{LayerNorm}(X + \text{sublayer}(X))$$

X: entrada original (skip connection)

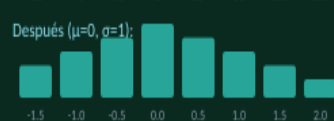
Layer Normalization + Conexión Residual — Estabilidad del entrenamiento

¿Qué hace Layer Normalization?

Antes (distribución raw):



Después ($\mu=0$, $\sigma=1$):



¿Por qué la conexión residual (skip)?

Sin skip: Gradiente se desvanece en redes profundas → imposible entrenar 96+ capas

Con skip: El gradiente fluye directamente → GPT-3 con 96 capas es posible

Bonus: Si la capa no aprende nada útil, $F(X)=0$ y $\text{output}=X$ (identidad)

Origen: He et al. (2015) — ResNet para visión. Transformers lo adoptan en 2017

⑥ Output Logits → Softmax → Token — Cómo el modelo genera texto

Linear (unembed)

El hidden state del último token se proyecta a vocab_size dimensiones: una puntuación (logit) por cada token posible.

Softmax

Convierte logits en probabilidades (suman 1.0). $\exp(\text{logit}_i) / \sum \exp(\text{logit}_j)$. Nunca cero.

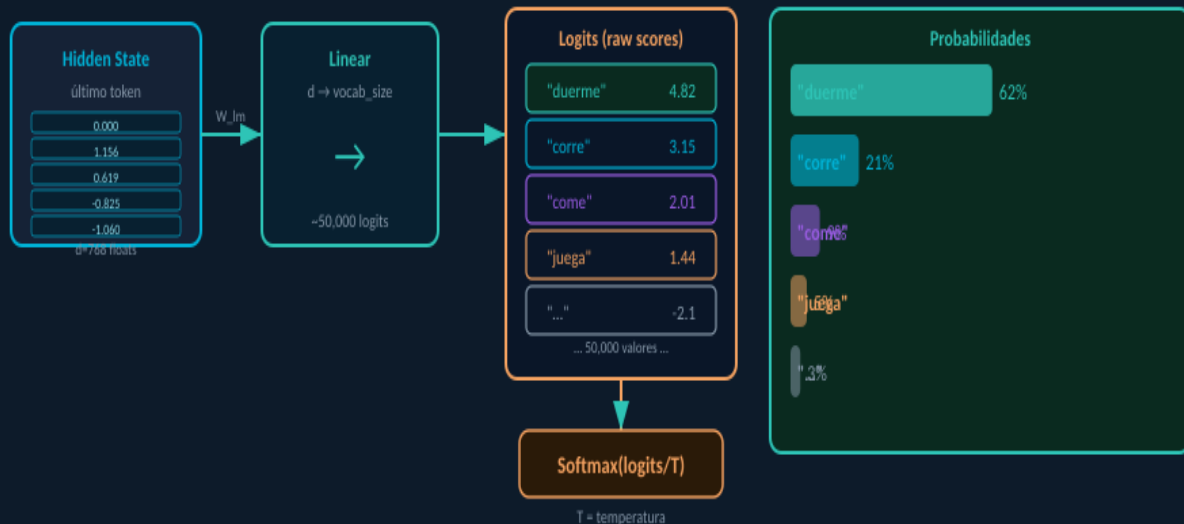
Temperatura

$T=1$: distribución natural. $T<1$: más conservador (texto seguro). $T>1$: más creativo (más errores).

Top-P (nucleus)

Suma probabilidades de mayor a menor hasta llegar a $P=0.9$. Muestra solo entre esos tokens. Muy usado en producción.

Output Logits → Softmax → Token → Generación autoregresiva



Generación Autoregresiva

1. Prompt → tokens de entrada
2. Forward pass → logits → sample → token
3. Agrega token al contexto → repite

Un token por vez → así genera "El gato duerme en..."

GPT-4o: ~100 tokens/seg → ~75 palabras/seg

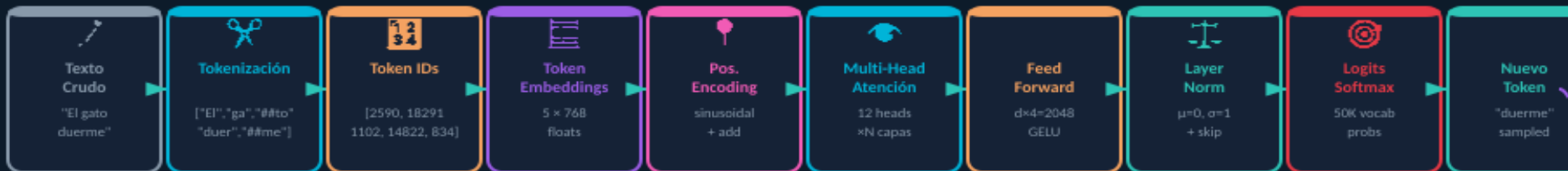
Estrategias de Sampling (cómo elegir el siguiente token)

Greedy: elige el más probable (determinista). Temp: T menor 1=conservador, T mayor 1=creativo.

Top-K: muestra entre los K tokens más probables ($K=50$). **Top-P**: acumula probs hasta $P=0.9$.

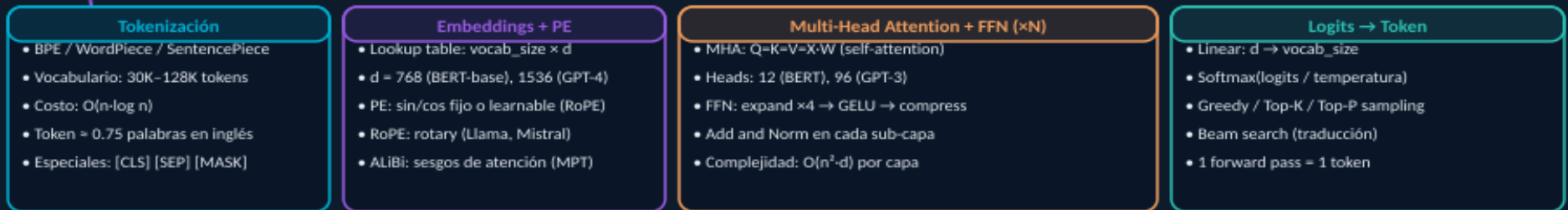
Pipeline Completo — Texto → Token → Embedding → Atención → Generación

Pipeline Completo: Texto → Token → Embedding → Atención → Generación



Este bloque se repite N veces (N=12 en GPT-2, N=96 en GPT-3)

Detalles técnicos de cada etapa



↪ Loop autoregresivo: el nuevo token se agrega al contexto y se repite

Impacto Real en la Industria



Sector Financiero

- Anti-fraude en tiempo real
- Análisis de riesgo crediticio
- Atención 24/7 automatizada
- KYC documental automático

↓40%
**costos
operativos**



Salud

- Resumen de historias clínicas
- Codificación ICD automática
- Asistente de diagnóstico
- Triage inteligente

↑35%
**eficiencia
médica**



Sector Público

- Chatbots de atención ciudadana
- Procesamiento de trámites
- Análisis de políticas públicas
- Transparencia gubernamental

×3
**más
trámites/dí
a**



Legal & Compliance

- Revisión de contratos
- Due diligence automatizado
- Monitoreo regulatorio
- Redacción de documentos

↓60%
**tiempo
revisión**

TEMA 2

Tipos de Modelos y Servicios LLM

Cloud, on-premise, open source — ¿Cuál es el correcto para tu caso de uso?

Arquitecturas de Modelos



Decoder-Only

Genera texto de izquierda a derecha, predice el siguiente token.

Modelos:

GPT-4, GPT-4o, Claude 3.x, Llama 3, Mistral, Gemini

Casos de uso:

- Generación de texto libre
- Chatbots y asistentes
- Código (Copilot, CodeLlama)
- Análisis y razonamiento

*Causal
(solo ve hacia atrás)*



Encoder-Only

Comprende el contexto completo (bidireccional). No genera texto.

Modelos:

BERT, RoBERTa, DistilBERT, XLM-R, mBERT

Casos de uso:

- Clasificación de texto
- Análisis de sentimiento
- Embeddings semánticos
- Búsqueda semántica (RAG)

*Bidireccional
(contexto completo)*



Encoder-Decoder

Combina comprensión (encoder) con generación (decoder).

Modelos:

T5, BART, mT5, Pegasus, MarianMT

Casos de uso:

- Traducción automática
- Resumen de documentos
- Q&A extractivo
- Corrección gramatical

*Bidireccional + Causal
(cross-attention)*

Cloud vs On-Premise | Casos LATAM

Criterio	 Cloud API	 On-Premise / Local
Costo por uso	Pay-per-token (predecible)	Infraestructura propia (upfront)
Privacidad	Datos van al proveedor	Datos 100% en tus servidores 
Calidad	Los mejores modelos (GPT-4o)	Depende del modelo descargado
Escalabilidad	Automática e instantánea	Limitada por hardware propio
Compliance	Requiere DPA/acuerdos	Control total sobre datos 
Latencia	10-60ms red + procesamiento	Local: <10ms en GPU dedicada
Mantenimiento	Ninguno (managed)	Actualización de modelos propia

Regla práctica para LATAM:

- Sector financiero/salud/gobierno → On-premise o Llama local (datos sensibles, GDPR/LGPD)
- Startups y MVPs → Cloud API (GPT-4o mini o Gemini Flash = menor costo de arranque)
- Empresas medianas con volumen alto → Modelo híbrido: local para datos sensibles + cloud para el resto

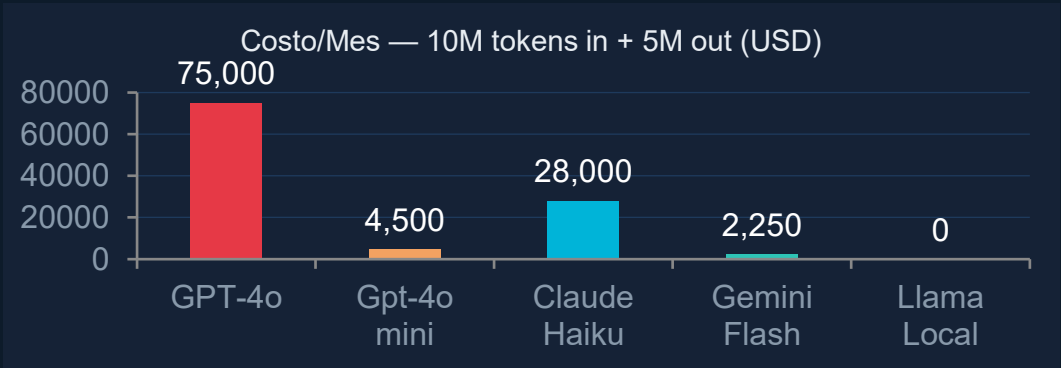
TEMA 3

Consideraciones de Costos

De la prueba de concepto a producción: cómo evitar sorpresas en la factura

Comparativa de Proveedores y Costos

Proveedor	Modelo	\$/1M tokens in	\$/1M tokens out	Fortaleza	Para LATAM
OpenAI	GPT-4o	\$2.50	\$10.00	Calidad máxima	Azure LATAM
OpenAI	GPT-4o mini	\$0.15	\$0.60	Mejor costo/calidad	Ideal MVPs
Anthropic	Claude 3.5 Haiku	\$0.80	\$4.00	Instrucciones complejas	Procesos regulados
Google	Gemini 1.5 Flash	\$0.075	\$0.30	Más económico	GCP Colombia/Mx
Local	Llama 3.2:3B	\$0.00	\$0.00	Privacidad total	Datos sensibles 



💡 Insight clave:

Gemini Flash vs GPT-4o:

97% de ahorro en tokens.

Ollama local:

Costo = \$0 por token
(solo electricidad + hardware)

ROI, Evaluación de Proveedores y Riesgo

Framework de TCO (Total Cost of Ownership)

Costos directos: Tokens, API calls, suscripciones

Infraestructura: GPU, servidores, cloud compute

Desarrollo: Ingeniería, prompt engineering

Operación: Monitoreo, mantenimiento, soporte

Datos: Preparación, storage, labeling

Compliance: Auditorías, DPA, certificaciones

Riesgos Críticos de Adopción

Vendor Lock-in

Migrar de GPT-4 a Claude puede requerir re-prompt. Usar LangChain como abstracción.

Hallucinations

Los LLM generan texto incorrecto con confianza. Implementar validaciones y RAG.

Costo runaway

Un loop de agentes puede generar millones de tokens en minutos. Siempre: rate limits.

Privacidad LGPD

Brasil LGPD, Colombia Ley 1581. Revisar DPA de cada proveedor antes de usar.



Demo en Vivo

1

LLM Local con Ollama

Ejecutar Llama 3.2 en tu laptop sin internet ni costo por token.

```
$ python  
examples/ollama_demo/demo_local.py
```

2

Streaming de Respuestas

Ver cómo los tokens llegan en tiempo real mejorando la UX.

```
$ python  
examples/ollama_demo/demo_streaming.py
```

3

API FastAPI + Ollama

Exponer el LLM como servicio REST profesional con Swagger UI.

```
$ make run-api →  
http://localhost:8000/docs
```

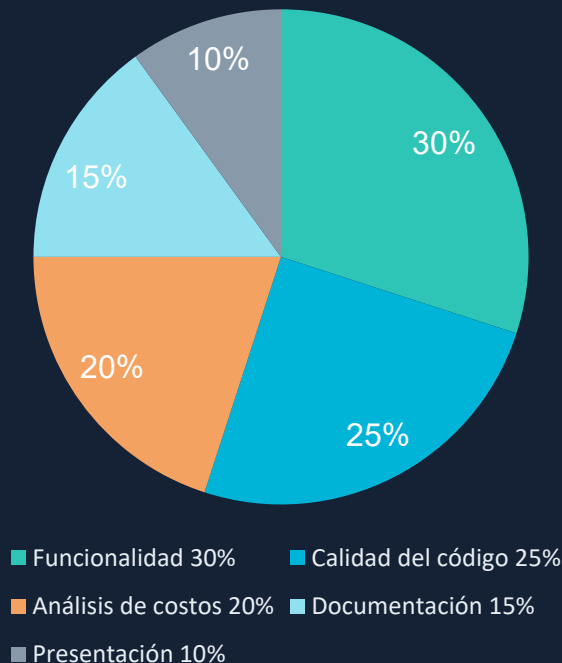
✓ Prerrequisitos: Ollama instalado + modelo descargado (ollama pull llama3.2:3b) + pip install -r requirements.txt

Proyecto Final — Entregables y Criterios de Evaluación

Entregables del Proyecto

1. Repositorio GitHub estructurado
 - README, requirements.txt, Makefile
 - Código documentado (docstrings)
2. API FastAPI funcional
 - Mínimo 3 endpoints documentados
 - Health check + Swagger UI
3. Integración LLM real
 - Al menos 1 proveedor (Ollama o API)
 - Streaming implementado
4. Análisis de costos
 - Comparativa de al menos 3 proveedores
 - Justificación de la selección

Criterios de Evaluación



Resumen y Próximos Pasos

✓ Aprendimos Hoy

- Transformers (2017) son la base de todos los LLM modernos
- 3 arquitecturas: decoder-only (GPT), encoder-only (BERT), encoder-decoder (T5)
- Cloud vs local: trade-off costo/privacidad/calidad
- Costos varían hasta 97% entre proveedores
- Ollama permite LLM local gratis en tu laptop

🚀 Para esta Semana

1. Instalar Ollama y ejecutar demo_local.py
2. Explorar la API FastAPI en /docs
3. Ejecutar la calculadora de costos con tu caso de uso real
4. Crear tu repositorio en GitHub con la estructura del Makefile
5. Identificar un caso de uso LATAM de tu sector para el proyecto